Hex Grid Search Ring Workflow

Purpose

`HexGridSearchRings` is a precomputed lookup table that stores, for each active hex cell, every other active cell within a configured ring distance. Each row answers this question:

- "From `HexGridCellId`, which `NearbyHexGridCellId` is reachable within `RingDistance` hex steps?"

This avoids recalculating ring expansions at request time.

Where the Table Is Defined

Entity model

- `BitSchedulerCore/HexGridSearchRing.cs:3`
- Columns:
  - `Id`
  - `HexGridCellId`
  - `NearbyHexGridCellId`
  - `RingDistance`

Navigation properties

- `BitSchedulerCore/HexGridCell.cs:29`
- `BitSchedulerCore/HexGridCell.cs:30`

Each `HexGridCell` has:

- `SearchRings`: rows where the cell is the origin
- `NearbySearchRings`: rows where the cell is the nearby result

EF Core mapping

- `BitSchedulerCore/Data/BitScheduleDbContext.cs:40`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs:289`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs:303`
- `BitSchedulerCore/Data/BitScheduleDbContext.cs:304`

Important details:

- `HexGridSearchRings` is a `DbSet<HexGridSearchRing>`
- `HexGridCellId -> HexGridCells.Id` cascades on delete
- `NearbyHexGridCellId -> HexGridCells.Id` uses `Restrict`
- Unique index on `(HexGridCellId, NearbyHexGridCellId)`
- Non-unique index on `(HexGridCellId, RingDistance)`

That indexing pattern matches the main usage: fetch all rows for one origin cell, already grouped by distance.

Migration that created the table

- `BitSchedulerCore/Migrations/20260619182643_AddHexGridTables.cs:115`
- `BitSchedulerCore/Migrations/20260619182643_AddHexGridTables.cs:181`
- `BitSchedulerCore/Migrations/20260619182643_AddHexGridTables.cs:187`
- `BitSchedulerCore/Migrations/20260619182643_AddHexGridTables.cs:192`

The migration creates:

- table `HexGridSearchRings`
- foreign keys back to `HexGridCells`
- indexes for origin/nearby uniqueness and origin/ring lookups

How the Grid Itself Is Generated

The search-ring table depends on the hex grid being generated first.

Generation options

- `BitSchedulerCore/Models/HexGridGenerationOptions.cs:17`
- `BitSchedulerCore/Models/HexGridServiceAreas.cs:16`

The default Edmonton configuration uses:

- `AreaName = EdmontonMetro`
- `HexRadiusMeters = 500`
- bounding box around the Edmonton metro area
- `MaxPrecomputedRingDistance = 8`

Cell generation

- `BitSchedulerCore/Services/HexGridGenerationService.cs:9`
- `BitSchedulerCore/Models/HexGridGenerationEngine.cs`

Workflow:

1. `GenerateGridAsync` calls `HexGridGenerationEngine.GenerateCells(options)`.
2. The engine converts the bounding box to axial hex coordinates.
3. It loops candidate `(q, r)` coordinates in that range.
4. It keeps only cells whose center point falls inside the configured lat/long bounds.
5. It stores those cells in `HexGridCells`, tied to a new `HexGridVersion`.
6. It also stores `MaxPrecomputedRingDistance` on the version record.

This means the ring table is built against a finite set of active persisted cells, not against theoretical coordinates.

How `HexGridSearchRings` Is Populated

Trigger points

The table is populated in two main places.

1. Application startup

- `BitScheduleServices/Infrastructure/ApiStartupInitializer.cs:59`
- `BitScheduleServices/Infrastructure/ApiStartupInitializer.cs:76`
- `BitScheduleServices/Infrastructure/ApiStartupInitializer.cs:144`

On startup, `EnsureHexGridAsync` does this:

- If no active hex grid version exists, it generates the Edmonton grid
- Then it builds neighbors
- Then it builds search rings
- If an active version already exists but has no search-ring rows, it builds them for that version

So a fresh environment can generate the entire table automatically during startup.

2. Feature/API workflow

- `BitScheduleServices/Features/HexGrid/HexGridFeatureService.cs:16`
- `BitScheduleServices/Features/HexGrid/HexGridFeatureService.cs:36`

There are explicit service methods to:

- generate a new Edmonton grid and immediately build its tables
- rebuild the tables for a specific grid version, optionally with a custom max ring distance

Table build service

- `BitSchedulerCore/Services/HexGridTableService.cs:31`
- `BitSchedulerCore/Services/HexGridTableService.cs:53`

`BuildSearchRingTableAsync(versionId, maxRingDistance)` works like this:

1. Load all active cells for the target grid version.
2. Find existing `HexGridSearchRings` rows whose `HexGridCellId` is one of those cell ids.
3. Delete those existing rows.
4. Rebuild the full set with `tableBuilder.BuildSearchRings(cells, maxRingDistance)`.
5. Save all rows.

Important implication:

- This is a full rebuild for the selected version's origin cells.
- It is not incremental.
- Increasing `maxRingDistance` means many more rows.

Actual row generation algorithm

- `BitSchedulerCore/Services/HexGridTableBuilder.cs:33`
- `BitSchedulerCore/Services/HexGridTableBuilder.cs:45`
- `BitSchedulerCore/Models/HexGridGeometry.cs:32`

For every active cell:

1. Build a dictionary of real persisted cells by `(Q, R)`.
2. Ask `HexGridGeometry.GetCoordinatesWithinDistance(cell.Q, cell.R, maxRingDistance)` for all axial coordinates in that radius.
3. For each generated coordinate:
    - If a real cell exists at that coordinate, create a `HexGridSearchRing` row
    - If no real cell exists, skip it
4. Store the hex distance as `RingDistance`
5. Sort by:
    - `HexGridCellId`
    - `RingDistance`
    - `NearbyHexGridCellId`

Two important details:

- Ring `0` is included, so every cell generates a self-row where `HexGridCellId == NearbyHexGridCellId` and `RingDistance == 0`.
- Boundary cells generate fewer rows because some coordinates within the radius fall outside the persisted grid and are skipped.

Why the Table Can Reach 2+ Million Rows

The growth is multiplicative:

- one origin row set per active cell
- one nearby row per valid cell within the configured radius

For a hex radius of `r`, the maximum number of coordinates within that radius is:

- `1 + 3r(r + 1)`

For the current default `MaxPrecomputedRingDistance = 8`:

- `1 + 3 * 8 * 9 = 217`

So each interior cell can contribute up to 217 search-ring rows.

Rough intuition:

- `cell count * about 217` rows for interior cells
- edge cells contribute less

That makes a 2+ million row table very plausible if the active Edmonton grid contains on the order of ten thousand cells.

How the Table Is Used at Runtime

Loaded into in-memory lookup

- `BitSchedulerCore/Services/HexGridLookupProvider.cs:14`
- `BitSchedulerCore/Services/HexGridLookupProvider.cs:52`
- `BitSchedulerCore/Services/HexGridLookupProvider.cs:88`

`HexGridLookupProvider.ReloadAsync()`:

1. Loads the active `HexGridVersion`
2. Loads all active `HexGridCells`
3. Loads all `HexGridNeighbors`
4. Loads all `HexGridSearchRings` for those origin cells
5. Builds an in-memory structure:
    - `RingIdsByCellId[cellId][ringDistance] = int[] nearbyCellIds`

This is the key reason the table exists: the app turns the precomputed rows into an in-memory dictionary for fast ring expansion.

Consumed by `HexGridSearchService`

- `BitSchedulerCore/Services/HexGridSearchService.cs:12`
- `BitSchedulerCore/Services/HexGridSearchService.cs:43`
- `BitSchedulerCore/Services/HexGridSearchService.cs:67`

Relevant methods:

- `GetGridId(latitude, longitude)`
    - converts a point to a hex cell id
- `GetGridIdsWithinRing(gridId, maxRingDistance)`
    - expands from ring 0 through `maxRingDistance`
- `ExpandSearch(gridId, startRing, endRing)`
    - returns lookup entries from the preloaded `RingIdsByCellId`

`ExpandSearch` does not query the database directly. It reads the lookup that was already built from `HexGridSearchRings`.

Exposed through feature endpoints

- `BitScheduleServices/Features/HexGrid/HexGridFeatureService.cs:81`

`GetHexGridRing` returns `searchService.GetGridIdsWithinRing(gridId, maxRingDistance)`, which is ultimately powered by the precomputed ring table.

Indirect usage in address resolution

- `BitScheduleServices/Features/Locations/AddressLocationService.cs:33`

Address resolution uses `GetGridId(latitude, longitude)` to assign a hex cell to a geocoded point. That path uses the cell lookup, but not the ring-expansion portion of the table directly.

What Does Not Currently Use It Much

There are model types suggesting a future resource-by-grid search workflow:

- `BitSchedulerCore/Models/ResourceGridSearchModels.cs`

Those types include:

- `ResourceGridSearchRequest`
- `ResourceGridSearchResult`
- `SearchRingDistance`

But in the current codebase I did not find an implemented service that uses those models to perform resource candidate searches from `HexGridSearchRings`. Right now the strongest concrete usage is:

- preload ring data into memory
- support ring expansion APIs
- support spatial lookup infrastructure around the active grid

End-to-End Workflow Summary

1. Startup or API code generates a `HexGridVersion` and many `HexGridCells`.
2. `BuildSearchRingTableAsync` deletes and rebuilds all search-ring rows for the active cells in that version.
3. `HexGridTableBuilder.BuildSearchRings` creates one row per reachable nearby cell within the configured max ring distance.
4. Startup then calls `HexGridLookupProvider.ReloadAsync()`.
5. The provider loads `HexGridSearchRings` into `RingIdsByCellId`.
6. `HexGridSearchService` uses that in-memory dictionary to answer ring queries quickly.

Key Takeaways

- `HexGridSearchRings` is a denormalized, precomputed expansion table.
- It is rebuilt in bulk, not maintained incrementally.
- The 2+ million row count is expected behavior for a large grid with `MaxPrecomputedRingDistance = 8`.
- The current code uses it mainly to build a fast in-memory ring lookup, not to execute ad hoc SQL ring searches per request.